

127. Word Ladder **Hard**

Acceptance: 44.7% | [LeetCode](#)

A **transformation sequence** from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words `beginWord` $\rightarrow$ `s1` $\rightarrow$ `s2` $\rightarrow$... $\rightarrow$ `sk` such that:

- Every adjacent pair of words differs by a single letter.
- Every `si` for $1 \leq i \leq k$ is in `wordList`. Note that `beginWord` does not need to be in `wordList`.
- `sk == endWord`

Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return *the number of words in the shortest transformation sequence from `beginWord` to `endWord`, or 0 if no such sequence exists.*

Example 1:

```
**Input:** beginWord = "hit", endWord = "cog",  
wordList = ["hot","dot","dog","lot","log","cog"]  
**Output:** 5  
**Explanation:** One shortest transformation sequence  
is "hit"  $\rightarrow$  "hot"  $\rightarrow$  "dog"  $\rightarrow$  "cog", which is  
5 words long.
```

Example 2:

```
**Input:** beginWord = "hit", endWord = "cog",  
wordList = ["hot","dot","dog","lot","log"]  
**Output:** 0  
**Explanation:** The endWord "cog" is not in wordList,  
therefore there is no valid transformation sequence.
```

Constraints:

- $1 \leq \text{beginWord.length} \leq 10$
- `endWord.length == beginWord.length`
- $1 \leq \text{wordList.length} \leq 5000$
- `wordList[i].length == beginWord.length`
- `beginWord`, `endWord`, and `wordList[i]` consist of lowercase English letters.
- `beginWord != endWord`
- All the words in `wordList` are **unique**.

Claude Opus Solution

Word Ladder - Complete Solution Guide

1. Intuition

The key insight is recognizing this as a **shortest path problem on an implicit graph**:

- Each **word** is a **node** in the graph.
- An **edge** connects two words if they differ by exactly **one letter**.
- We want the **shortest path** from `beginWord` to `endWord`.

Since all edges have equal weight (1 step), **BFS** is the perfect tool—BFS explores nodes level-by-level, guaranteeing that the first time we reach `endWord`, we've found the shortest path.

Critical optimization: Instead of comparing every word against every other word ($O(N^2)$ edges), we generate "neighbors" on-the-fly by changing each letter to all 26 possibilities and checking if the result exists in the word set ($O(1)$ lookup).

2. Approach (Pseudocode)

1. Put all words in a `HashSet` for $O(1)$ lookups
2. If `endWord` not in set $\rightarrow$ return 0 (impossible)
3. Initialize BFS queue with (`beginWord`, `level=1`)
4. While queue not empty:
 - a. Pop current word and its level
 - b. If word == `endWord` $\rightarrow$ return level
 - c. For each position `i` in the word:
 - For each letter 'a' to 'z':
 - Form `new_word` by replacing `word[i]` with letter
 - If `new_word` is in the set:
 - * Remove it from set (mark visited, avoids revisits)
 - * Add (`new_word`, `level + 1`) to queue
5. Return 0 (`endWord` unreachable)

3. Python Solution

```
from collections import deque
from string import ascii_lowercase
from typing import List

class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str]) -> int:
        # Convert wordList to a set for O(1) membership checks.
        # This set also doubles as our "unvisited" tracker:
        # removing a word marks it as visited.
        word_set = set(wordList)

        # If endWord isn't reachable (not in dictionary), no sequence exists.
        if endWord not in word_set:
            return 0

        # BFS queue: each element is (current_word, sequence_length_so_far).
        # We start counting at 1 because beginWord itself counts as the
        # first word in the transformation sequence.
        queue = deque([(beginWord, 1)])

        while queue:
            # Pop from the FRONT (FIFO) – this is what makes it BFS.
            # BFS guarantees we process words in order of increasing distance.
            word, level = queue.popleft()

            # If we've reached the target, this is the SHORTEST path
            # because BFS reaches nodes by minimum number of steps.
            if word == endWord:
                return level

            # Generate all possible "one-letter-different" words.
            # For each position in the word...
            for i in range(len(word)):
                # ...try every lowercase letter as a replacement.
                for ch in ascii_lowercase:
                    # Build the candidate neighbor by swapping one char.
                    # word[:i] + ch + word[i+1:] replaces position i.
                    new_word = word[:i] + ch + word[i + 1:]

                    # Only valid if it's an actual dictionary word AND unvisited.
                    if new_word in word_set:
                        # Remove from set so we never enqueue it again.
                        # This prevents cycles and redundant work – the first
                        # time we reach a word via BFS is always the shortest.
                        word_set.remove(new_word)

                        # Enqueue with incremented level (one more transformation).
                        queue.append((new_word, level + 1))

        # Queue exhausted without finding endWord -> no valid sequence.
        return 0
```

4. Complexity Analysis

Let **N** = number of words in `wordList`, **M** = length of each word.

Time Complexity: $O(N \cdot M^2)$

- We process each word **at most once** (it's removed from the set when visited) $\rightarrow$ **N** iterations.
- For each word, we examine **M** positions $\times$ **26** letters = **26M** candidates.
- Building each new string via slicing costs **O(M)**.
- Total: $N \times 26M \times M = O(N \cdot M^2)$ (the constant 26 is dropped).

Space Complexity: $O(N \cdot M)$

- The `word_set` stores up to **N** words, each of length **M** $\rightarrow O(N \cdot M)$.
- The BFS queue holds up to **N** words in the worst case $\rightarrow O(N \cdot M)$.

- Each `new_word` string built is **$O(M)$** , but transient.
- Total: **$O(N \cdot M)$** .

5. Alternative Approaches

A. Bidirectional BFS (Faster in practice)

Search **simultaneously** from `beginWord` and `endWord`, alternating to always expand the **smaller** frontier. When the frontiers meet, we've found the path.

- **Why faster?** A single BFS explores $\sim b^d$ nodes (branching factor b , depth d). Bidirectional explores $\sim 2 \cdot b^{(d/2)}$, which is exponentially smaller.
- Same asymptotic worst case, but dramatic real-world speedup.

```
def ladderLength(self, beginWord, endWord, wordList):
    word_set = set(wordList)
    if endWord not in word_set:
        return 0

    begin, end = {beginWord}, {endWord}
    level = 1

    while begin and end:
        # Always expand the smaller frontier (optimization).
        if len(begin) > len(end):
            begin, end = end, begin

        next_level = set()
        for word in begin:
            for i in range(len(word)):
                for ch in ascii_lowercase:
                    nxt = word[:i] + ch + word[i+1:]
                    if nxt in end: # Frontiers met!
                        return level + 1
                    if nxt in word_set:
                        next_level.add(nxt)
                        word_set.discard(nxt) # mark visited
        begin = next_level
        level += 1
    return 0
```

B. Precomputed Pattern Graph (the `my_solution.py` approach)

Build a graph using **wildcard patterns**: `"hot"` maps to `"*ot"`, `"h*t"`, `"ho*"`. Words sharing a pattern are neighbors.

- **Tradeoff**: Faster neighbor lookup if you query many times, but more upfront memory ($O(N \cdot M)$ patterns). Not worth the overhead for a single query.

C. The Standard BFS (recommended) ✓

The solution above strikes the best balance of **simplicity, correctness, and efficiency** for this problem. The pattern—**implicit graph + BFS for shortest path with unit edges**—is the core takeaway.

Key Patterns to Remember

1. "Shortest path / minimum steps" + "unweighted graph" → **BFS**
2. **Implicit graph**: don't build edges explicitly; generate neighbors on demand.
3. **Mark visited by removing from set** to save space and prevent cycles.
4. **Bidirectional BFS** when both source and target are known.

Codex Solution

1. Intuition

This problem is a **shortest path problem in an unweighted graph**.

Think of each word as a node.

There is an edge between two words if they differ by exactly one character.

For example:

```
hit -> hot
hot -> dot
dot -> dog
dog -> cog
```

Since every transformation has equal cost, the shortest transformation sequence can be found using **Breadth-First Search**, BFS.

However, instead of explicitly building the whole graph, we can generate neighbors on the fly by changing each character to 'a' through 'z'.

To optimize further, we can use **bidirectional BFS**:

- Start one BFS from `beginWord`
- Start another BFS from `endWord`
- Always expand the smaller frontier
- Stop when the two searches meet

This reduces the search space significantly.

2. Approach

Key idea

At each step, for the current word, try changing every character to every lowercase letter.

For example, for "hit":

```
ait, bit, cit, ..., hot, ..., hzt
hat, hbt, hct, ...
hia, hib, hic, ...
```

If the generated word exists in `wordList`, it is a valid next transformation.

Pseudocode

```
function ladderLength(beginWord, endWord, wordList):
    wordSet = set(wordList)

    if endWord not in wordSet:
        return 0

    front = {beginWord}
    back = {endWord}

    remove beginWord and endWord from wordSet if present

    length = 1

    while front and back:
        if front is larger than back:
            swap front and back

        nextFront = empty set

        for word in front:
            for each index i in word:
                for each character c from 'a' to 'z':
                    newWord = word with word[i] replaced by c

                    if newWord is in back:
                        return length + 1
```

```

        if newWord is in wordSet:
            remove newWord from wordSet
            add newWord to nextFront

    front = nextFront
    length += 1

return 0

```

3. Python Solution

```

from typing import List
from string import ascii_lowercase

class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str]) -> int:
        # Store all dictionary words in a set for O(1) lookup.
        word_set = set(wordList)

        # If endWord is not in the dictionary, transformation is impossible.
        if endWord not in word_set:
            return 0

        # Bidirectional BFS frontiers.
        # One starts from beginWord, the other starts from endWord.
        front = {beginWord}
        back = {endWord}

        # Avoid revisiting these words.
        word_set.discard(beginWord)
        word_set.discard(endWord)

        # The sequence currently contains beginWord itself.
        length = 1

        # Continue while both search frontiers are non-empty.
        while front and back:
            # Always expand the smaller frontier.
            # This is the main optimization of bidirectional BFS.
            if len(front) > len(back):
                front, back = back, front

            # Store the next level of BFS.
            next_front = set()

            # Try all one-letter transformations from every word in current frontier.
            for word in front:
                word_chars = list(word)

                for i in range(len(word_chars)):
                    original_char = word_chars[i]

                    # Replace position i with every lowercase letter.
                    for ch in ascii_lowercase:
                        if ch == original_char:
                            continue

                        word_chars[i] = ch
                        new_word = "".join(word_chars)

                        # If the newly generated word is in the opposite frontier,
                        # both BFS searches have met.
                        if new_word in back:
                            return length + 1

                    # If this is a valid unvisited dictionary word,
                    # add it to the next BFS level.
                    if new_word in word_set:
                        word_set.remove(new_word)
                        next_front.add(new_word)

```

```

        # Restore the original character before moving to next position.
        word_chars[i] = original_char

    # Move to the next BFS level.
    front = next_front
    length += 1

# If the searches never meet, no valid transformation exists.
return 0

```

4. Complexity Analysis

Let:

- N be the number of words in `wordList`
- M be the length of each word
- Alphabet size is fixed at 26

Time Complexity

For each visited word, we try:

```
M positions * 26 letters
```

Each generated word creation costs $O(M)$ because `"".join(word_chars)` creates a string of length M .

So the worst-case time complexity is:

```
 $O(N * M * 26 * M) = O(N * M^2)$ 
```

Since 26 is constant:

```
 $O(N * M^2)$ 
```

Bidirectional BFS usually performs much faster in practice because it searches from both ends and expands the smaller frontier.

Space Complexity

We store:

- `word_set`, containing up to N words
- BFS frontiers, containing up to N words in the worst case

Therefore:

```
 $O(N)$ 
```

If counting the storage of the actual strings, it is:

```
 $O(N * M)$ 
```

5. Alternative Approaches

1. Standard BFS

We can start BFS only from `beginWord`.

For each word, generate all possible one-letter transformations and check if they are in the dictionary.

This is simpler, but often slower than bidirectional BFS.

```

from collections import deque
from string import ascii_lowercase
from typing import List

class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str]) -> int:
        word_set = set(wordList)

        if endWord not in word_set:

```

```

    return 0

    queue = deque([(beginWord, 1)])
    word_set.discard(beginWord)

    while queue:
        word, length = queue.popleft()

        for i in range(len(word)):
            for ch in ascii_lowercase:
                new_word = word[:i] + ch + word[i + 1:]

                if new_word == endWord:
                    return length + 1

                if new_word in word_set:
                    word_set.remove(new_word)
                    queue.append((new_word, length + 1))

    return 0

```

Time complexity is also:

$O(N * M^2)$

Space complexity:

$O(N)$

2. Precomputed Pattern Graph

Another approach is to preprocess wildcard patterns.

For example:

```

hot -> *ot, h*t, ho*
dot -> *ot, d*t, do*
dog -> *og, d*g, do*

```

Words sharing the same pattern are neighbors.

This makes neighbor lookup easier, but requires extra preprocessing space.

Time complexity is roughly:

$O(N * M^2)$

Space complexity:

$O(N * M)$

This approach is useful when many neighbor queries are needed.